FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
FAST School of Computing · Karachi Campus · Department of Software Engineering
AL-2002 Artificial Intelligence (Laboratory) · Fall 2026

PROJECT PROPOSAL

Semester Lab Project · Weightage 1 mark

Group Information

	Name	Roll Number	Section	Role in the project
Leader	Saniya Asim	24K-3084	BSE-5B	Game engine, rules and unit tests, tournament runner, repository
Member 2	Aamna	24K-3001	BSE-5B	Rule-based agents, Pygame interface, human player
Member 3	Aisha Shaikh	24K-3026	BSE-5B	Expectimax agent, evaluation function, charts and analysis

Project Title	LudoMax: Expectimax AI for Ludo under Dice Chance
Project Idea	Idea No. **09** from the Project Ideas list ☒ Own idea (needs approval) ☐
Category	Part A — Game ☒ Part B — Applied / Data ☐
GitHub Repository URL	SANIYA-ASIM/LudoMax-Expectimax-AI
Date of Submission	22 October 2026

1. Project Title and Summary

Title: LudoMax: Expectimax AI for Ludo under Dice Chance

LudoMax is a Ludo game where computer players choose which piece to move after the dice is rolled. Ludo depends on luck, but a roll can allow two to four possible moves. The best move depends on whether a piece can leave base, capture an opponent, stay safe, or get closer to the finish. The project will include a full Ludo rules engine and a Pygame board. One person can play against one to three computer players. The Expectimax player looks ahead and calculates the average result for all six possible dice values, which helps it judge risk better than fixed if-then rules. We will compare it with aggressive, defensive, and random players over many simulated games to measure how much strategy helps in a game of chance.

2. Problem Statement and Motivation

Ludo has four pieces per player moving around a 52-square loop. A dice roll can allow up to four moves, and rolling a six gives another turn. There are so many possible piece positions that checking every possible future move is not practical. The game is also uncertain because no player controls the dice. A move that seems safe may still allow an opponent to capture a piece on their next turn. Safe squares, capturing, needing a six to leave base, and rolling the exact number to finish all affect the choice. Casual players often use simple rules, such as “always capture” or “keep pieces together,” but these rules are not always tested. Our project will use win rates to measure whether a careful AI strategy works better than simple rules.

3. Problem Formulation

3.1 PEAS Description

Component	Description for your project
Performance measure	Win by getting all four pieces home first. Measure win rate, captures made and lost, and turns needed to win.
Environment	Ludo board with a 52-square shared loop, four private home columns, eight safe squares, 2–4 players, and a six-sided die. Our own engine will run the rules.
Actuators	Choose which piece to move for the rolled number, then show the move on the Pygame board.
Sensors	Positions of all 16 pieces, dice value, current player, extra-roll status, consecutive-six count, and safe-square locations.

3.2 Environment and Agent Type

Property	Classification — tick one
Observability	Fully observable ☐ Partially observable ☒
Determinism	Deterministic ☐ Stochastic ☒ (dice)
Change over time	Static ☒ Dynamic ☐
State and actions	Discrete ☒ Continuous ☐
Agents	Single-agent ☐ Multi-agent / adversarial ☒
Agent type	Simple reflex ☐ Model-based ☐ Goal-based ☐ Utility-based ☒ Learning ☐

3.3 Problem Representation

Element	Your representation
State representation	GameState stores each piece's progress (4 pieces per player): −1 means in base, 0–50 means on the loop, 51–56 means in the home column, and 57 means finished. It also stores the current player, extra-roll status, and consecutive-six count.
Actions / moves	move(piece i, dice d), where i is 0–3. A move is allowed only when the rules permit it, such as needing a six to leave base and not moving past the finish. If no move is legal, the turn passes.
Initial state	All pieces start in base, and player 0 rolls first. Dice are generated using a seeded random.Random.
Goal or terminal test	The game ends when one player has all four pieces at progress 57. That player wins.
Cost, utility, fitness or evaluation	At the end, a win scores +1000 and a loss scores −1000. Otherwise, a heuristic score is used: our score minus the best opponent score

4. Proposed AI Approach

#	Technique	Role in the project	Own code or library
1	Expectimax search (depth-limited; MAX, chance and opponent nodes)	Main AI: chooses which piece to move by averaging the results of the six possible dice values. It also counts how many search nodes it visits.	Own code
2	Aggressive rule-based agent	Captures if possible; otherwise leaves base, then moves the leading piece forward.	Own code
3	Defensive rule-based agent	Protects the piece in the most danger, prefers safe squares, and avoids risky positions.	Own code

B	**Baseline:** random legal-move agent	Baseline for comparison. A fair win rate is 1/N for N players.	Own code (seeded random)
–	Statistics and charts	Calculates averages, 95% confidence intervals, and plots. Pygame is used for the interface.	Libraries: NumPy, Matplotlib, Pygame

Ludo has uncertain dice results, several players, and a board everyone can see. Expectimax suits this kind of game. A MAX node chooses our move, a chance node considers each dice result with a 1/6 chance, and opponent nodes choose the reply that is worst for us. In games with three or four players, all opponents are treated as one opponent. One dice roll and move count as one search step because there can be about 4 × 6 = 24 choices at each step. At the depth limit, the AI calculates our score minus the best opponent score. The score gives points for progress, finished pieces, pieces outside base, and pieces on safe squares. It subtracts points for danger, based on the chance that an opponent can capture a piece on their next roll. We will tune the score weights using different random seeds from those used in the final tests.

4.1 Planned Class Design

- **Board** — stores constants such as the 52-square loop, safe squares, start positions, and home columns. It can convert a piece's progress to a board square, check safe squares, and find pieces on a square.
- **Move (dataclass)** — stores the piece, dice value, starting and ending progress, and any captured piece.
- **GameState** — stores piece progress, current player, extra-roll status, and consecutive sixes. It checks legal moves, applies moves, checks game end and winner, and copies a state.
- **Game (environment)** — manages the seeded random generator, board, game state, and players. It rolls the dice, plays turns, and runs a game.
- **Agent (abstract)** — defines the player name and choose_move(state, dice). RandomAgent, AggressiveAgent, and DefensiveAgent each choose a move. ExpectimaxAgent stores depth, weights, and nodes expanded, and runs expectimax, chance_value, and evaluate. HumanAgent returns the piece clicked in the interface.
- **GameUI (Pygame)** — draws the board and dice handles clicks
- **Tournament** — runs agents through a set number of games with a seed, rotates seats, and saves a results table as CSV.

5. Scope

In scope — the features we commit to delivering.

#	Feature	Why it is essential
1	Ludo engine using OOP: leave base on a six, captures, safe squares, extra turn on a six, lose turn after three sixes, exact roll to finish, and 2–4 players.	Every player and experiment uses this engine, so the rules must be correct and tested.
2	Random, Aggressive, and Defensive players.	Provides a baseline and the other strategies required by the project guidelines.
3	Expectimax player with dice chance nodes, a scoring function, and adjustable search depth.	This is the main AI method for the project.
4	Pygame board where a person plays against 1–3 AI players, with a console menu as a backup.	Needed for the project demonstration and viva.
5	Experiment runner with seeded tournaments, a results table, at least three charts, and one command to repeat the experiment.	Shows AI performance and supports the experiment marks.

Out of scope — deliberately excluded.

Online multiplayer; team Ludo; blockade rules where pieces are stacked; other house-rule versions; learning-based players such as MCTS, reinforcement learning, or neural networks; learning individual human habits; animations, sound, or polished graphics; and web or mobile versions. These features are excluded so a three-person group can finish the game engine, three player types, and a proper experiment on time.

6. Data or Environment Source

Source name and URL	No external source. Games are generated by our own Ludo engine; dice come from Python's random.Random(seed) with one seed per game.
Type	Public dataset ☐ Web API ☐ Self-collected ☐ Generated / simulated ☒
Access requirements	No key needed ☒ Free key required ☐ (Paid sources are not permitted)
Approximate size	About 100 games per matchup, one CSV row per game (a few hundred KB in total); no raw data to download.
Licence / terms of use	Own code and generated results; libraries used: Pygame (LGPL), NumPy and Matplotlib (BSD-style).
Offline fallback	Everything runs offline after pip install. Results are cached in outputs/results/*.csv so charts can be redrawn without re-running the simulations.

7. Evaluation Plan

What will be compared	Expectimax (depth 1, 2, 3) against Aggressive, Defensive and Random, in 2-player matches and in 4-player free-for-all games; also the effect of search depth on strength and cost.
Metric(s)	Win-rate with 95% confidence interval; average captures made and suffered; average turns per game; nodes expanded and time per decision.
Baseline	Random legal-move agent (fair win-rate 50% with 2 players, 25% with 4 players).
Test instances / test set	Game seeds 0–99 for every matchup (50 games for depth 3), with seats rotated so that no agent always moves first. Heuristic weights are tuned on separate seeds 9000–9049.
Number of runs and seed	100 games per matchup; master seed 2002, game i uses seed 2002 + i.
Planned charts (at least three)	(1) Win-rate by agent in 4-player games with error bars; (2) win-rate of Expectimax versus search depth; (3) nodes expanded and time per move versus depth; (4) head-to-head win-rate matrix; (5) captures made versus suffered per agent.

8. Division of Work

Member	Modules / components owned	Approximate share
Saniya Asim (Leader)	Board, GameState, Move, Game (engine and rules); unit tests; Tournament and experiment runner; README and repository set-up.	34%
Aamna	Agent base class; Random, Aggressive and Defensive agents; Pygame GameUI and HumanAgent; console menu fallback.	33%
Aisha Shaikh	ExpectimaxAgent and evaluation function; weight tuning; depth experiments; charts and analysis.	33%

All	Proposal and report sections on our own modules; testing and checking each other's code.	–

9. Milestones and Timeline

#	Milestone — what will be demonstrably working	Target date	Owner
1	Engine plays complete random-vs-random games for 2–4 players from the console; unit tests pass for capture, safe square, exact finish and three sixes.	27 Oct	Saniya Asim
2	Aggressive and Defensive agents finish a 100-game seeded tournament against Random and print win-rates.	3 Nov	Aamna
3	Expectimax at depth 1–2 beats Random over 100 two-player games and reports nodes expanded.	10 Nov	Aisha Shaikh
4	Pygame board is playable end to end: a human against 1–3 AI opponents.	17 Nov	Aamna
5	One command reproduces the results table and at least three charts in outputs/; weights tuned on separate seeds.	24 Nov	Saniya, Aisha
6	Experiments complete, final integration, documentation and repository cleanup (fresh-clone test).	27 Nov	All

10. Expected Outcome and Risks

Running python src/main.py will open a Pygame menu. The user can choose one to three AI opponents and play on a board with a dice display. Running python src/experiments.py will create the results table and charts again. We expect Expectimax at depth 2 to win more often than a fair share (about 35–45% in four-player games against one of each other player type), while random play is expected to finish last. Luck may reduce the difference.

Risk 1: Depth 3 may run too slowly in Python. We will limit four-player search to depth 2, save repeated positions, and test depth 3 on fewer games.

Risk 2: Dice luck may hide differences between players. We will run more games for the main comparison without displaying the board and report confidence intervals.

For office use — Lab Instructor	
Proposal status	Approved ☐ Approved with changes ☐ Revision required ☐
Comments	
Signature and date	

End of Project Proposal